PCP Assignment 1 - 2026
Fireline: Parallel Heat diffusion for a Wildfire simulation
RNJNEE001

1) Introduction:

This project investigates the effects of parallelisation and threads compared to the serial running of a complex wildfire simulation using the java shared memory (fork/join) framework. It models heat diffusion, cooling, fuel consumption and fire spreading (combustion) across a 2d grid of either mixed or grass terrain using a stencil based algorithm. We are given a serial model of the program and tasked with implementing a parallel model using fork/join framework. We then compare the two using identical simulation, benchmarking convergence times across different grid sizes and recording the speedup that parallelisation provides. In addition, we also use a test to determine the appropriate serial cutoff, and lastly, running these benchmarks on an external machine to test whether our results hold.

2) Methods:

2.1) "Divide and Conquer" approach:
The parallel implementation uses fork/join framework by making 2 changes. First we make the task for each thread to run, called FireTask. FireTask represents the computation of the rectangular region given to each thread at each timestep of the simulation. If the number of cells in the task exceeds the sequential cutoff, it divides the current region into 2 subregions, creating a firetask for each region. One subtask is executed i.e the leftTask.fork() while the other is computed i.e rightTask.compute(). Lastly join() is used to wait for the forked task to complete.

```java
//compute method
@Override
protected FireMap.StepResult compute() {
    int rowCount = rowEnd - rowStart;
    int columnCount = columnEnd - columnStart;
    //tried usign int but for some reason didnt work, then i ragequit so chatbot said use long. dont even ask
    long numberOfCells = (long) rowCount * columnCount;

    // basic aaah thing that goes in every slide in the notes
    if (numberOfCells <= sequentialCutoff) {
        return map.updateRegion(mode,rowStart,rowEnd,columnStart,columnEnd);
    }

    FireTask LeftTask;
    FireTask RightTask;

    // split along the longer side so we havee more balanced regions
    if (rowCount >= columnCount) {
        int middleRow = rowStart + rowCount / 2;
        // look suspiciously similar to class test 1 icl cough cough
        LeftTask = new FireTask(map,mode,rowStart,middleRow,columnStart,
                columnEnd,sequentialCutoff);
        RightTask = new FireTask(map,mode,middleRow,rowEnd,columnStart,
                columnEnd,sequentialCutoff);

    } else {
        int middleColumn = columnStart + columnCount / 2;
        LeftTask = new FireTask(map,mode,rowStart,rowEnd,columnStart,
                middleColumn,sequentialCutoff);
        RightTask = new FireTask(map,mode,rowStart,rowEnd,middleColumn,
                columnEnd,sequentialCutoff);
    }
    //same as every attendanc quiz
    LeftTask.fork();
    FireMap.StepResult secondResult = RightTask.compute();
    FireMap.StepResult firstResult = LeftTask.join();

    return FireMap.StepResult.combine(firstResult, secondResult);
}
```

This recursive action continues until the task is small enough to be processed sequentially.

2.2) Avoiding Race Conditions

FireMapParallel uses separate current-state and next-state grids. During each timestep, all tasks read from the unchanged current grid and write their results to the next grid. FireTask assigns each task a non-overlapping region of the next grid, preventing concurrent writes to the same cell. After all tasks complete, join() ensures the next grid is fully updated before it becomes the current state for the following timestep.

```
public StepResult stepParallel(Mode mode) {

    // Prepare the next buffers so that neighboring cells can be read from the current buffer and written to the next buffer.
    prepareNextState();
    FireTask Task = new FireTask(this,mode,rowStart: 1,rows - 1,columnStart: 1,columns - 1,sequentialCutoff);
    pool.execute(Task);
    StepResult result = Task.join();
    //same as serial, call after all grid regions have been updated to swap the current and next buffers and update the step count.
    completeStep();
    return result;
}
}
```

2.3) Determining sequential cutoff

The sequential cutoff was determined by testing different cutoffs on a fixed configuration of the parallel program. The config was as follows

Grid size	seed	wildfire/diffusion	Terrain type	Sequential Cutoffs
500 x 500	17	wildfire	mixed	{500,1000,2500 5000,10 000,20 000, 40 000, 80 000}

To begin with, a baseline runtime of the serial program was run 7 times, of which the median was taken and compared to against the median of 7 iterations of each sequential cutoff. The speedup was then measured and reported into a csv file. Upon completing the experiment, it seems that 20 000 is the most appropriate (fastest) sequential cutoff.

2.4) Validating the solution: The parallel solution was evaluated by comparing it to the serial solution. We check whether the following are for both parallel and serial solutions under teh same configuration:Number of Timesteps,Convergence,Final Burning cells,Number of cells burned,Maximum peak temperature,Final maximum change.

We run this 5 times, and if all 5 tests pass (they did) we know that the parallel solution works.

2.5) Benchmarking: The benchmarking was done with 3 bash scripts. One that checks the correctness, one that checks the sequential cutoffs, and one that checks the actual speedup benchmarks. Each of these scripts creates a csv file with the results.

2.5.1) Benchmark justifications:

For the Correctness benchmark: Five deterministic cases were used, varying grid size, seed, simulation mode and landscape. The serial and parallel versions were compared using

convergence results and all three generated images to demonstrate correctness beyond a single run.
For the Sequential Cutoff: A fixed 500×500 wildfire case (seed 17, mixed landscape, tolerance 0.05) was used so that only the cutoff affected performance. Cutoffs from 500 to 80,000 cells were tested using two warm-ups and seven timed repetitions, with the median core simulation time used to select the best cutoff.
Speedup Benchmark: Speedup was measured for grid sizes from 100×100 to 1500×1500 while keeping the seed, mode, landscape and convergence settings fixed. This range captures small cases dominated by parallel overhead, medium cases where parallelisation becomes beneficial, and larger cases where scaling begins to decline.

2.6) Machine specs:

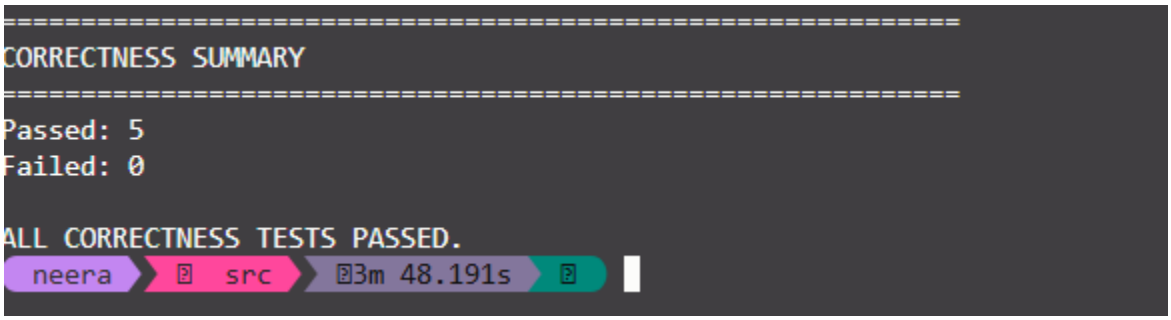
Device	**CPU**	**Java**	**Cores**
Laptop	**i7-12650H**	**OpenJDK 21, release 11**	**10 physical 16 logical**
Nightmare	**Xeon E5620 2.4ghz**	**Java/Javac 17, release 11**	**4 physical 6 logical**

2.7) Warm up and timing method

Two warm-up runs were performed before timing. Each configuration was then run seven times, with the median core simulation time used as the representative result. Timing used System.nanoTime() around the simulation loop only, excluding image generation and file writing.

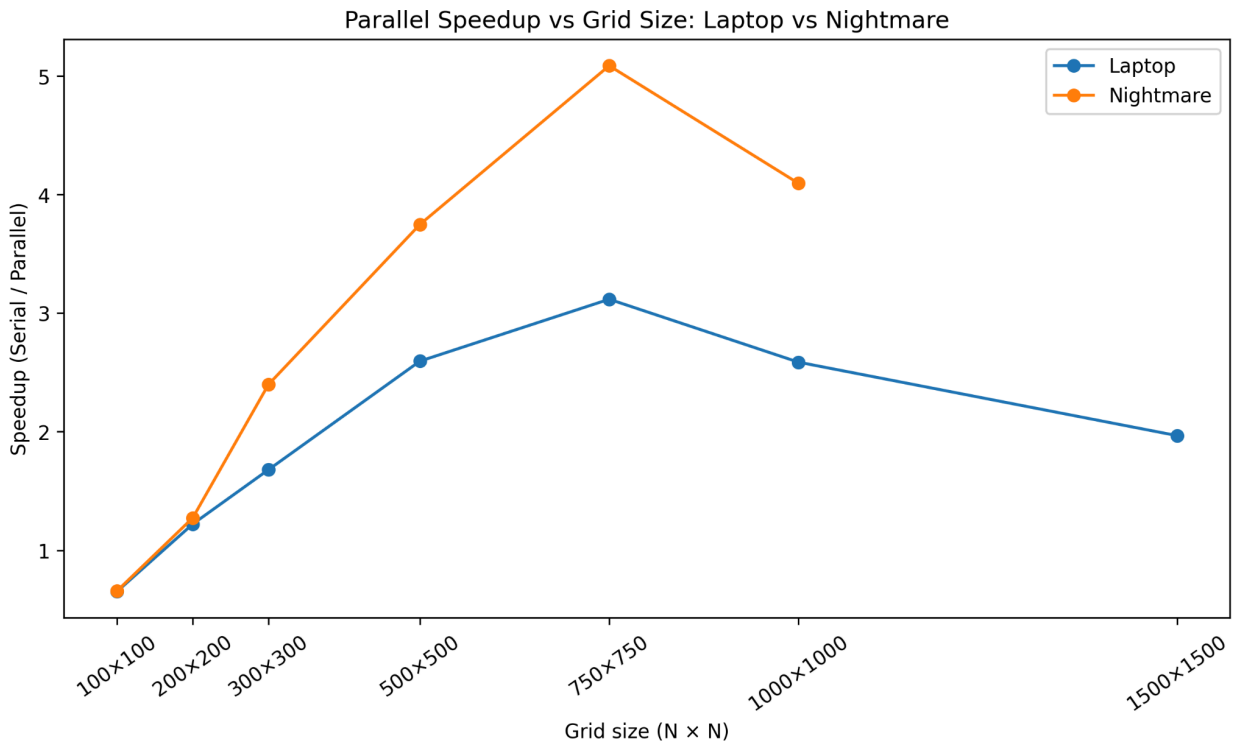
3) Results and Discussion

3.1) Equivalent results:



3.2)Speedup graphs

Fig 1. Speedup on local machine vs nightmare



The graph shows that the parallel solution achieved its greatest speedup at approximately a 750×750 grid size, after which speedup declined as the grid size increased. This suggests that increasing the workload does not necessarily result in greater speedup, potentially because system-level limitations become increasingly significant at larger problem sizes. At 100×100, the speedup is below 1×, meaning that the parallel implementation was slower than the serial version. This is likely due to the overhead associated with parallel execution, including task creation, scheduling, forking and joining, outweighing the benefits of parallelism for such a small workload. Nightmare achieved noticeably higher speedups than my local machine despite this being unexpected based on the apparent machine specifications. I was unable to determine the precise cause of this difference, so it remains an unexplained performance anomaly. The 1500×1500 case was not measured on Nightmare because of the lengthy execution time. The maximum observed speedup on Nightmare was 5.09×, compared with an ideal maximum of approximately 6× based on its six logical processors , whereas, the laptop achieved a maximum speedup of approximately 3.12×, compared with an ideal maximum of approximately 16× based on its 16 logical processors.

3.3) Simulation configurations

For the main speedup experiment:

Seed	Mode	Landscape	Tolerance	Max Steps	Cutoff	size
17	wildfire	miced	0.05	50000	20 000	100x100 200x200 300x300

						500x500 1000x1000 1500x1500

Which is exactly the requirements given benchmark.

For correctness:

We deliberately chose random configs in seeds, mode, and grid to ensure correctness across each parameter. The 5 tests were

For cutoff: As discussed in the methods

3.4) Convergence Timesteps:

size	**100x100**	**200x200**	**300x300**	**500x500**	**750x750**	**1000x1000**	**1500x1500**
#timesteps	**1025**	**1827**	**2951**	**4528**	**6932**	**8917**	**23836**

Sequential cutoff

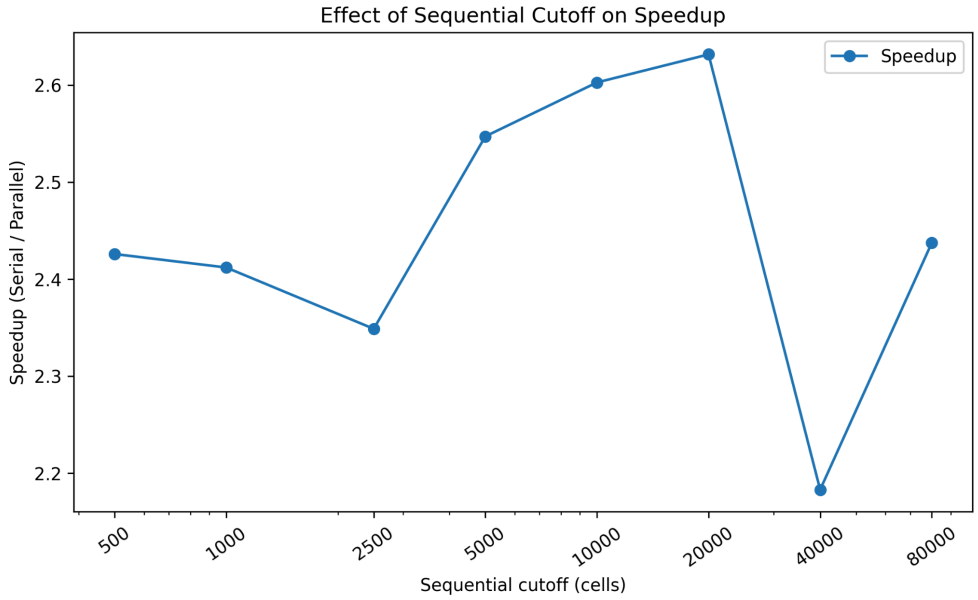


Fig1 Seq cutoff

From these results, a sequential cutoff of 20 000 clearly produced the highest speedup. It suddenly drops after 20 000 and seems to spike up again at 40 000, but with less speedup.

Thus, 20 000 was used as the sequential cutoff in all the benchmark tests.

Conclusion

The results show that parallelisation is worthwhile for medium and large Fireline simulations, but not for very small grids where Fork/Join overhead dominates. The parallel implementation

achieved a maximum speedup of 5.09× at 750×750 on Nightmare, while the 100×100 case was slower than the serial version. Correctness was confirmed across multiple grid sizes, seeds, modes and landscapes, with matching simulation results and identical output images. The cutoff experiment also showed that performance depends on task granularity, with 20,000 cells giving the best result for the tested 500×500 configuration. Overall, parallelisation provides substantial performance benefits once the workload is large enough to outweigh parallel overhead.

AI Declaration:
Ai was used in this assignment only in making the 3 bash scripts. THe code in the bash scripts for testing was fully made by ChatGPT and was only used for testing purposes.

Validity:
All reported benchmark values were generated directly from the supplied Java programs and saved by the benchmark scripts. Raw console outputs, CSV results and generated images were retained in the src/results directory for verification.